

Subtitle: Practical, Industry-Style Q&A across NumPy, Pandas, Statistics, EDA, Feature Engineering & Machine Learning

Date: 2026

This is a curated set of **100 data science interview questions** that are actually asked across product and services companies for candidates with **0–3 years of experience**. Companies in scope include TCS, Infosys, Wipro, Accenture, Cognizant, Mu Sigma, Fractal, Tiger Analytics, ZS Associates, EXL, Genpact, Flipkart, Swiggy, Zomato, PhonePe, Paytm, Walmart Labs, Razorpay, and similar.

What's covered

#	Topic	Questions	Range
1	NumPy	12	Q1 – Q12
2	Pandas	18	Q13 – Q30
3	Statistics	15	Q31 – Q45
4	Exploratory Data Analysis (EDA)	15	Q46 – Q60
5	Feature Engineering	15	Q61 – Q75
6	Machine Learning	25	Q76 – Q100
	Total	100	

- Time-series modelling (ARIMA, Prophet, etc.)
- Recommender systems
- Heavy mathematical derivations, calculus, or matrix proofs

- Deep learning specifics (CNN/RNN/Transformer architectures)

How to use this document

1. **Read actively.** Read the question first. Try to answer it out loud before reading the explanation.
2. **Speak in 60–90 seconds.** Most answers in this guide are tuned to a 1–2 minute speaking length.
3. **Notice the pitfalls.** Each answer flags common mistakes — interviewers love probing those.
4. **Practice the code.** For NumPy and Pandas, run the snippets in a notebook so the syntax becomes muscle memory.
5. **Tie everything to a project.** Be ready to relate concepts to a real project, internship, or capstone.

Conventions used

- `python` blocks contain runnable example code.
- **Bold** highlights interview-critical terms.
- *Italics* mark common follow-up questions interviewers may ask.
- "Pitfall" sections list mistakes you must avoid.

Table of Contents

- [About This Document](#)
- [Section 1 – NumPy \(Q1–Q12\)](#)
- [Section 2 – Pandas \(Q13–Q30\)](#)
- [Section 3 – Statistics \(Q31–Q45\)](#)
- [Section 4 – Exploratory Data Analysis \(Q46–Q60\)](#)
- [Section 5 – Feature Engineering \(Q61–Q75\)](#)
- [Section 6 – Machine Learning \(Q76–Q100\)](#)
- [Final Notes for the Candidate](#)

Section 1 – NumPy (Q1–Q12)

NumPy is the foundation of the Python data stack. Interviewers expect speed-of-light answers on dtypes, broadcasting, copies vs views, and why vectorization is fast. Expect at least 2–3 NumPy questions in any data role screen.

Q1. What is the difference between a NumPy ndarray and a Python list?

Both store sequences, but they are very different under the hood. A Python list stores pointers to scattered Python objects on the heap, and each element can be a different type. A NumPy ndarray stores raw values in one contiguous block of memory with a single fixed dtype, like `int64` or `float32`. Because of this layout, NumPy can run vectorized operations as compiled C loops, which is typically 10x to 100x faster than equivalent loops over a list. Memory is also tighter; a million floats in a list takes around 28 MB, but in an ndarray only about 8 MB. Lists are flexible and good for dynamic appends, but they do not support broadcasting, multi-dimensional slicing, or axis-wise aggregations. In practice I use a list when I am building data row by row, then convert to an ndarray once the size is known, because appending to an ndarray creates a new array each time and is expensive.

Q2. Explain `dtype` in NumPy and why it matters.

`dtype` is the data type of every element in an ndarray. Since arrays are homogeneous, the dtype is fixed at creation. Common ones are `int32`, `int64`, `float32`, `float64`, `bool`, and `object`. By default integers become `int64` and floats become `float64` on a 64-bit system. Dtype matters for three reasons. First, **memory**: `float32` uses half the RAM of `float64`, which is significant when you load lakhs of rows or push data to a GPU. Second, **overflow**: an `int8` array adding 100 to 200 silently wraps to a negative number, which can corrupt features. Third, **speed**: `float32` ops are often faster on modern hardware. When you mix dtypes, NumPy upcasts to the widest common type, so `int + float` gives `float`. In ML pipelines I usually downcast to `float32` for model inputs and use `astype` with explicit casting rather than relying on implicit promotion, to avoid surprises.

Q3. What does the `axis` argument do in NumPy aggregations?

`axis` tells NumPy which dimension to collapse during a reduction. The simple rule I remember is, **axis is the one that disappears**. For a 2D array of shape `(2, 3)`, `sum` with `axis=0` collapses the rows and gives a result of shape `(3,)`, which is the column sums. `axis=1` collapses columns and gives shape `(2,)`, the row sums. `axis=None`, the default for many functions, reduces over everything and returns a scalar. For a 3D image array of shape `(H, W, C)`, `axis=2` aggregates across channels. A useful trick is `keepdims=True`, which keeps the reduced axis as size 1 so the result still broadcasts back against the original. For example, to mean-center each row I write `A - A.mean(axis=1, keepdims=True)`. Without `keepdims` that subtraction would either fail or align the wrong way.

Q4. Explain different ways of indexing and slicing NumPy arrays.

NumPy supports several indexing styles. **Basic slicing** like `a[1:5]` or `a[:, 0]` returns a view that shares memory. **Multi-dimensional slicing** uses commas per axis, so `a[1:3, 2:4]` picks a sub-block. **Fancy indexing** passes a list or array of integers, like `a[[0, 2], [1, 3]]`, and returns a copy. **Boolean indexing** uses a mask of the same shape, like `a[a > 5]`, and also returns a copy. You can mix them, for example `a[mask, 0]`. A common bug is writing `a[mask][:] = 0` expecting it to update `a`, but the inner indexing already returned a copy, so the assignment is lost. The correct form is `a[mask] = 0` in one step. Another point candidates miss is the difference between `a[0, :]` which has shape `(4,)`, and `a[0:1, :]` which keeps the row dimension and has shape `(1, 4)`.

Q5. What is the difference between `shape`, `reshape`, `ravel`, and `flatten`?

`shape` is an attribute giving the size along each axis as a tuple. `reshape` returns the same data viewed with a new shape, as long as the total element count matches; you can pass `-1` to let NumPy infer one dimension. `ravel` returns a 1D **view** of the array if memory is contiguous, so it is cheap and modifications propagate back. `flatten` always returns a 1D **copy**, so it is safer but uses extra memory. The gotcha interviewers love is that `reshape` is not always a view. If the array is non-contiguous, for example after a transpose, `reshape` silently returns a copy. So `a.T.reshape(...)` usually allocates new memory. For predictable behaviour I use `ravel` when I want speed and know I will not need a copy, `flatten` when I want isolation, and `reshape` when I am restructuring known-contiguous data.

Q6. Explain broadcasting in NumPy with an example.

Broadcasting is how NumPy does element-wise operations on arrays of different shapes without copying or looping. The rules compare shapes from the right. Two dimensions are compatible if they are equal or one of them is 1, and missing leading dimensions are treated as 1. The smaller array is virtually stretched along the size-1 axes. For example, a matrix `A` of shape `(3, 4)` plus a vector `b` of shape `(4,)` works because `b` is treated as `(1, 4)` and stretched to `(3, 4)`. If I want to add a column vector instead, I reshape it to `(3, 1)` using `b[:, None]`. The big win is that no real copy happens, so it is fast and memory cheap. A typical use case is mean-centering: `X - X.mean(axis=0)`. The classic error is mixing shapes `(3,)` and `(4,)`, which raises a broadcasting `ValueError` because neither dimension is 1 or equal.

Q7. What is the difference between a copy and a view in NumPy?

A **view** shares memory with the original array, so modifying one changes the other. A **copy** is independent. Basic slicing such as `a[1:5]` gives a view, while fancy indexing and boolean masking always give copies. You can force a deep copy with `a.copy()`, and check sharing with `np.may_share_memory(a, b)`. This matters for two reasons. **Performance** is one; views are $O(1)$ since no data is moved. The bigger reason is **correctness**. If a function takes an array and silently mutates a slice, the caller's data changes too, which causes subtle bugs that are hard to trace. I have seen pipelines where standardizing a slice quietly rescaled the original training set. The flag `arr.flags.owndata` tells you whether the array owns its data. When in doubt, especially when passing slices into functions that modify them, take an explicit copy.

Q8. How does `np.where` work and when do you use it?

`np.where` has two forms. With three arguments, `np.where(condition, x, y)` returns an array taking values from `x` where the condition is `True` and from `y` otherwise. With one argument, `np.where(cond)` returns the indices where the condition holds. It is essentially **vectorized if-else**, much faster than a Python loop with branching. A common use is clipping, like `np.where(a > 0, a, 0)` to zero out negatives. For feature engineering, you can bucket a column with nested `where` calls, but for more than two branches I prefer `np.select` with a list of conditions and choices, which is cleaner. A point interviewers probe: the difference between `np.where` and boolean indexing. **Boolean indexing extracts** elements that match the mask, returning a smaller array. `np.where` **preserves shape and substitutes** values, which is what you want when building features that align row by row.

Q9. What is the difference between `np.concatenate`, `np.vstack`, `np.hstack`, and `np.stack`?

All four combine arrays but in different ways. `np.concatenate` joins along an **existing** axis; the inputs must match in every dimension except that axis. `np.vstack` stacks vertically and behaves like `concatenate` on `axis=0`, also promoting 1D arrays to rows first. `np.hstack` stacks horizontally; for 1D arrays it joins them end to end, and for 2D it concatenates on `axis=1`. `np.stack` is different from all of these because it creates a **new axis**. So stacking two arrays of shape `(3, 4)` with `np.stack` gives `(2, 3, 4)`, while `concatenate` on `axis=0` gives `(6, 4)`. In practice I use `vstack` and `hstack` for quick row or column joins of feature blocks, `concatenate` when I am explicit about the axis, and `stack` when I want to add a batch or sample dimension. Mismatched shapes raise `ValueError`, so always print shapes before joining.

Q10. Why is vectorization preferred over Python loops in NumPy, and is `np.vectorize` actually fast?

Vectorization means expressing operations on whole arrays rather than looping element by element. NumPy executes these inside compiled C with contiguous memory and SIMD-friendly access, so the Python interpreter overhead per element is gone. On a million floats, a Python loop can take seconds while the vectorized form runs in milliseconds. It is also more readable: `c = a * b + 1` is clearer than a `for` loop with `append`. The trade-off is memory; vectorized code can create intermediate arrays, so for very large data I use in-place forms like `np.add(a, b, out=a)` or move to Numba. A common interviewer trap is `np.vectorize`. Despite the name, it is just a syntactic wrapper around a Python loop and gives almost no speedup. If you really need a custom scalar function on an array, write the math with NumPy primitives or use Numba's `@jit`, **not** `np.vectorize`.

Q11. How do you generate reproducible random numbers in NumPy?

The modern way is the **Generator API** introduced in NumPy 1.17. You create a generator with a seed and call methods on it, which keeps the random state local to that object instead of using global state.

```
import numpy as np
rng = np.random.default_rng(42)
rng.normal(0, 1, size=(3, 3))
rng.integers(0, 10, size=5)
```

The legacy `np.random.seed` plus `np.random.rand` still works but is global, so any library that calls `np.random` can reset it and break reproducibility. Reproducibility matters for ML experiments, train-test splits, and debugging, because the same seed gives the same outputs and reviewers can replay your run. A few caveats: results are reproducible within the same NumPy version and platform, but not always across hardware. In a full ML pipeline I also seed Python's `random` module and the framework, like `torch.manual_seed`, otherwise data shuffling or weight init still varies.

Q12. How do you handle NaN values in NumPy arrays?

NaN is a special float that **propagates through arithmetic**, so any sum or mean that touches a NaN becomes NaN. Two rules to remember. First, never compare with `==`, because `np.nan == np.nan` is `False`; always use `np.isnan` to get a boolean mask. Second, NaN exists only for float dtypes, so an integer column with missing values gets upcast to float when loaded. For aggregations, NumPy provides NaN-aware versions like `np.nanmean`, `np.nansum`, `np.nanstd`, and `np.nanmax` that

skip NaNs. To replace them, I use `np.nan_to_num` with a fill value, or assign through a mask: `a[np.isnan(a)] = 0`. To count missing values per column in a feature matrix, `np.isnan(X).sum(axis=0)` is the standard one-liner. For mixed object arrays or pandas DataFrames I switch to `pd.isna` because it also handles `None` and `NaT`, which `np.isnan` does not.

Section 2 – Pandas (Q13–Q30)

Pandas is the most-tested library in DS interviews. Expect questions that mix concept (`loc` vs `iloc`, `apply` vs vectorization) with code (`groupby`, `merge`, `pivot_table`). 90% of analytics work happens here.

Q13. What is the difference between a Pandas Series and a DataFrame?

A **Series** is a 1-D labelled array, like a single column with an index, and it holds one dtype. A **DataFrame** is a 2-D labelled table, basically a dict of Series sharing the same row index, where columns can have different dtypes.

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
df = pd.DataFrame({'age': [25, 30], 'city': ['Pune', 'Delhi']})
```

Selecting one column from a DataFrame like `df['age']` returns a Series. A DataFrame has both `.index` and `.columns`, while a Series has only `.index`. A common follow-up is *what does `df.iloc[0]` return?* It returns a Series representing the row, with column names as its index. A pitfall to watch is that mixing Series and DataFrame in operations can broadcast unexpectedly, so always check shapes with `.shape` and dtypes with `.dtypes` before arithmetic.

Q14. Which parameters of `read_csv` do you commonly tune in real projects?

The ones I tune most are `sep` for tab or pipe delimited files, `usecols` to read only required columns and save memory, `dtype` to force dtypes and avoid `object` where not needed, `parse_dates` to parse date columns at load time, `na_values` to treat tokens like `'NA'`, `'-'`, `'?'` as NaN, `nrows` and `skiprows` to preview large files, `chunksize` to iterate huge files, and `encoding` like `'utf-8'` or `'latin-1'` for messy files.

```
df = pd.read_csv('sales.csv', usecols=['id', 'amt', 'dt'],
                 parse_dates=['dt'], dtype={'id': 'int32'})
```

A common pitfall is loading a 5 GB file without specifying `dtype` or `usecols`, which leads to OOM. I always sample with `nrows=1000` first to inspect the schema, then load the full file with the right parameters.

Q15. Explain `loc` vs `iloc` with a concrete example.

`.loc` is **label-based**, using index labels and column names, and it is **end-inclusive** in slices. `.iloc` is **position-based**, using integer positions from 0 to `n-1`, and is **end-exclusive** like Python slicing.

```
df = pd.DataFrame({'a': [1, 2, 3]}, index=['x', 'y', 'z'])
df.loc['x': 'y', 'a']    # rows x and y
df.iloc[0:2, 0]          # rows 0 and 1
```

Use `.loc` when you have meaningful index labels like dates or IDs, and `.iloc` for positional slicing such as `df.iloc[-1]` for the last row. A pitfall is that `df.loc[df['col']>5]` works for boolean masking, but `df.iloc[df['col']>5]` does not, because `iloc` needs integer positions, not booleans. A common follow-up is what `df[0:2]` does — on a DataFrame it slices rows by position, but on a Series with non-integer index it slices by label, so prefer `loc` or `iloc` explicitly.

Q16. How do you filter a DataFrame using boolean conditions and `isin`?

Boolean masking is the standard idiom in Pandas:

```
df[df['age'] > 25]
df[(df['age']>25) & (df['city']=='Pune')]
df[df['city'].isin(['Pune', 'Delhi', 'Mumbai'])]
df[~df['city'].isin(['Pune'])]
```

Key points: use `&`, `|`, `~`, **not** `and`, `or`, `not`, and parentheses around each condition are mandatory because of operator precedence. `isin` is much cleaner than chaining many `==` with `|`. For substring matching, use `df[df['name'].str.contains('raj', case=False, na=False)]`. The `na=False` argument avoids errors when the column has NaNs. A common follow-up is filtering

between values, which is `df[df['age'].between(25, 35)]` and is inclusive by default. This pattern shows up in almost every interview SQL-to-Pandas translation question.

Q17. When would you use `df.query()` instead of boolean indexing?

`query()` lets you write filters as a string expression, which is more readable for complex conditions:

```
df.query('age > 25 and city == "Pune"')
threshold = 30
df.query('age > @threshold')
df.query('city in ["Pune","Delhi"]')
```

Advantages are cleaner multi-condition filters with no repetitive `df[...]`, slightly faster execution on large frames using `numexpr` when available, and it works well with column names that are valid identifiers. Limitations are that column names with spaces need backticks like `df.query('`order amount` > 100')`, and you cannot easily call arbitrary methods inside the string. For one-off simple filters, regular boolean indexing is fine. A pitfall is that `query` returns a copy, so modifying the result will not change the original – combine with `.loc` for assignment.

Q18. What is the difference between `apply`, `map`, and `applymap`?

`map` works on a Series and maps each value via a function, dict, or another Series – great for value substitution. `apply` works on a Series or DataFrame, and on a DataFrame it applies a function along an axis, row or column. `applymap` was element-wise on a DataFrame but is deprecated since 2.1 and replaced by `DataFrame.map`.

```
df['city'].map({'Pune':1, 'Delhi':2})
df['age'].apply(lambda x: x*2)
df.apply(lambda col: col.max() - col.min())
df.apply(lambda row: row['a']+row['b'], axis=1)
df.map(lambda x: str(x).strip())
```

A pitfall is that `apply(axis=1)` is slow because it iterates row-wise in Python, so always check whether a vectorized alternative exists. A common follow-up is which is faster, `map` or `apply` – `map` with a dict is essentially a hash lookup and is the fastest of the three.

Q19. Why is vectorization preferred over `apply` in Pandas?

Vectorized operations push the loop down to C and NumPy, while `apply` typically loops in Python, which is orders of magnitude slower on large frames.

```
# Slow
df['total'] = df.apply(lambda r: r['price']*r['qty'], axis=1)

# Fast
df['total'] = df['price'] * df['qty']

# String vectorization
df['email_lower'] = df['email'].str.lower()

# Conditional vectorization
import numpy as np
df['band'] = np.where(df['age']>=18, 'adult', 'minor')
```

Use arithmetic operators, `.str`, `.dt`, `np.where`, `np.select`, and `pd.cut` instead of `apply` whenever possible. On 1M+ rows, vectorization can be 50–500x faster. A pitfall is people benchmarking on 1k rows where the difference is invisible, so always test on realistic sizes. If logic is genuinely complex, `apply` is acceptable, or use `numba` for hot paths.

Q20. Walk through `groupby` with multiple aggregations.

`groupby` splits the frame, applies a function per group, and combines results.

```
df.groupby('city')['amount'].sum()

# Multiple aggregations on multiple columns
df.groupby('city').agg({
    'amount': ['sum', 'mean'],
    'orders': 'count'
})

# Named aggregations (recommended for clean column names)
df.groupby('city').agg(
    total_amt=('amount', 'sum'),
    avg_amt=('amount', 'mean'),
```

```
n_orders=('orders','count')
).reset_index()
```

Pass a list of columns to group by multiple keys, like `df.groupby(['city','category'])`. Use `as_index=False` or `.reset_index()` to flatten the group keys back into columns. `agg` accepts strings, functions, or lambdas. A pitfall is that NaN keys are dropped by default, so use `dropna=False` to keep them. A common follow-up is `count` vs `size` — `count` ignores NaN per column while `size` returns the row count of the group regardless of NaN.

Q21. How do `transform`, `agg`, and `filter` differ within `groupby`?

All three operate per group but return different shapes. `agg` reduces each group to a single row or scalar per column. `transform` returns an output the same length as input, broadcasting the group result back, which is great for adding group-level features. `filter` returns a subset of original rows where the group satisfies a condition.

```
# agg: one row per city
df.groupby('city')['amount'].sum()

# transform: each row gets its city's mean
df['city_mean'] = df.groupby('city')['amount'].transform('mean')

# filter: keep only cities with >100 rows
df.groupby('city').filter(lambda g: len(g) > 100)
```

A pitfall is that `transform` requires the function to return a scalar or same-length array per group, otherwise it errors. Use `transform` for feature engineering tasks like deviation from group mean, group rank, or share-of-total computations — these come up very often in case-study rounds.

Q22. Compare `pivot`, `pivot_table`, and `melt`.

`pivot` reshapes long to wide and requires unique index-columns combinations, otherwise it errors. `pivot_table` is like `pivot` but aggregates duplicates via `aggfunc`, default `mean`, and is what you actually use in real data. `melt` reshapes wide to long, the opposite of `pivot`.

```
# wide -> long
long = df.melt(id_vars=['id'], value_vars=['jan','feb','mar'],
               var_name='month', value_name='sales')
```

```
# long -> wide with aggregation
wide = df.pivot_table(index='city', columns='month',
                      values='sales', aggfunc='sum', fill_value=0)
```

`pivot_table` supports multiple `values` and multiple `aggfunc` like `['sum', 'mean']`, and `margins=True` adds totals. A pitfall is forgetting `fill_value`, which leaves NaNs for missing combinations. A common follow-up is when you *melt* – when data comes wide with one column per month or year, and you need a tidy long format for plotting or `groupby`.

Q23. Explain `merge` with the four join types and how it differs from `concat`.

`merge` is SQL-style join on keys, while `concat` stacks frames along an axis.

```
pd.merge(left, right, on='id', how='inner')
pd.merge(left, right, on='id', how='left')
pd.merge(left, right, on='id', how='right')
pd.merge(left, right, on='id', how='outer')

# Different key names
pd.merge(orders, users, left_on='user_id', right_on='id')

# Stack vertically or horizontally
pd.concat([df1, df2], axis=0, ignore_index=True)
pd.concat([df1, df2], axis=1)
```

`df.join` is shorthand for index-based merge. Use `indicator=True` to see which side each row came from, with values `left_only`, `right_only`, or `both`. `validate='1:m'` enforces cardinality and catches bad joins early. A pitfall is many-to-many joins exploding row counts – always check `len()` before and after, because duplicate keys silently inflate metrics.

Q24. How do you detect and handle missing values?

For detection I use `df.isna().sum()` to get nulls per column, `df.isna().mean()*100` for percentage missing, and `df[df['x'].isna()]` to inspect rows where a column is null.

Handling depends on context:

```
df.dropna()
df.dropna(subset=['x'])
df.dropna(thresh=3)

df['age'].fillna(df['age'].median())
df['city'].fillna('Unknown')
df['x'].interpolate()
```

Drop if missingness is rare and random, impute if dropping loses signal. For ML pipelines I use `SimpleImputer` so the same logic runs at train and inference. A pitfall is that `np.nan != np.nan`, so `df['x']==np.nan` does not work — always use `.isna()`. A common follow-up is NaN vs None vs NaT: NaN for floats, NaT for datetime, and None becomes NaN in numeric columns but stays None in object columns.

Q25. How do you find and remove duplicates?

```
df.duplicated()
df.duplicated(subset=['email'])
df.duplicated(keep='first')

df.drop_duplicates(subset=['email'], keep='last')
df['email'].nunique()
df['email'].is_unique
```

`duplicated` returns a Series of booleans, and combining it with filtering lets you inspect duplicates. For example `df[df.duplicated(subset=['email'], keep=False)]` shows all duplicate rows, not just the extras. Specify `subset` carefully because duplicates by full row versus by business key are different things. `keep=False` is useful when investigating every duplicated record. A pitfall is that trailing whitespace, casing, and trailing dots in emails create false-unique entries, so always normalize first using `df['email'] = df['email'].str.strip().str.lower()`. A common follow-up is counting duplicates, which is `df.duplicated().sum()`.

Q26. What does `value_counts` do and what are its useful options?

`value_counts` returns counts of unique values in a Series sorted descending — the go-to for categorical EDA.

```
df['city'].value_counts()
df['city'].value_counts(normalize=True)*100
df['city'].value_counts(dropna=False)
df['city'].value_counts().head(10)

# Numerical column with bins
df['age'].value_counts(bins=5)

# Multiple columns combinations
df[['city', 'category']].value_counts()
```

`normalize=True` gives proportions, useful for class imbalance checks. `dropna=False` is critical when you suspect missing values you would otherwise miss. It returns a Series indexed by unique values, so you can chain `.head()` or `.plot.bar()`. A pitfall is case sensitivity – `'Pune'` and `'pune'` are different, so normalize text first. A common follow-up is the difference vs `groupby().size()` – same counts, but `value_counts` is sorted by count and works directly on a Series.

Q27. How do you sort, and what are `nlargest` / `nsmallest` used for?

```
df.sort_values('amount', ascending=False)
df.sort_values(['city', 'amount'], ascending=[True, False])
df.sort_values('amount', na_position='first')
df.sort_index()

# Top-N without sorting the whole frame
df.nlargest(10, 'amount')
df.nsmallest(5, 'price')
df.nlargest(10, ['amount', 'date'])
```

`nlargest` and `nsmallest` are faster than `sort_values().head(n)` because they use partial sorting, roughly $O(n)$ versus $O(n \log n)$. They preserve the original index, useful for keeping references back to the source. For groupwise top-N use `df.groupby('city').apply(lambda g: g.nlargest(3, 'amount'))`. A pitfall is the default `ascending=True` for `sort_values` – it is easy to forget when looking for highest values, so use `nlargest` to make intent obvious. A common follow-up is stable sort, which uses `kind='mergesort'`.

Q28. How do you optimize memory of a Pandas DataFrame using dtypes and category?

Default dtypes like `int64`, `float64`, and `object` are wasteful. Common optimizations:

```
df.info(memory_usage='deep')

# Downcast numerics
df['id'] = pd.to_numeric(df['id'], downcast='integer')
df['price'] = pd.to_numeric(df['price'], downcast='float')

# Convert low-cardinality strings to category
df['city'] = df['city'].astype('category')
df['category'] = df['category'].astype('category')

df['active'] = df['active'].astype('bool')
```

`category` is dramatic for columns with few unique values relative to rows, like 5 cities across 10M rows – it stores integers plus a lookup. It also helps `groupby` and `merge` speed. A pitfall is making a high-cardinality column like `user_id` with millions of uniques categorical, since overhead exceeds savings. Set dtypes at `read_csv` time using the `dtype` parameter to avoid loading the bloated version first. Category hurts when you constantly add new values, since appending unknown categories triggers expensive conversions.

Q29. How do you read and write Parquet, and why prefer it over CSV?

Parquet is a columnar binary format that is much faster and smaller than CSV.

```
df.to_parquet('sales.parquet', engine='pyarrow', compression='snappy')
df = pd.read_parquet('sales.parquet')

# Read only required columns – huge speedup
df = pd.read_parquet('sales.parquet', columns=['id', 'amount', 'dt'])

# Partitioned write for big data
df.to_parquet('sales/', partition_cols=['year', 'month'])
```

Why I prefer Parquet for analytics: it preserves dtypes including datetime, so no re-parsing is needed; files are typically 5–10x smaller than CSV; reads are much faster especially with column pruning; and it

works seamlessly with Spark, Athena, and BigQuery. CSV is fine for small files or when humans need to open them. A pitfall is that `pyarrow` or `fastparquet` must be installed. For huge data I partition by date columns and read only the needed partitions, which avoids loading everything into memory.

Q30. What is the `SettingWithCopyWarning` and how do you avoid chained indexing?

The warning appears when you assign to a slice that may be a view or a copy, because Pandas cannot guarantee the original is being modified.

```
# Bad: chained indexing
df[df['age']>25]['salary'] = 100000    # may not update df

# Good: use .loc in a single call
df.loc[df['age']>25, 'salary'] = 100000

# When deriving a sub-frame, copy explicitly
sub = df[df['city']=='Pune'].copy()
sub['flag'] = 1
```

The rule is to use `.loc[row_mask, col]` for assignment, never `df[mask][col] = ...`. A pitfall is ignoring the warning, which silently produces bugs where your dataframe does not actually change. I treat it as an error, not a warning, and fix it immediately because debugging silent data corruption later is much worse.

Section 3 – Statistics (Q31–Q45)

These questions are deliberately **non-mathy**. interviewers probe whether you can *explain* statistical thinking to a business stakeholder – not whether you can prove the Central Limit Theorem. Practice saying these out loud.

Q31. What is the difference between population and sample, and why does it matter in practice?

Population is the complete set you want to study, like all Swiggy users in India. A **sample** is a subset you actually collect, say 10,000 users. We rarely access the full population because it is too costly or

impossible, so we compute sample statistics like mean and standard deviation and use them to estimate population parameters.

This matters because every model you build at work is trained on a sample. If the sample is biased or too small, your conclusions will not generalize. Confidence intervals, p-values, and standard errors all exist because sample estimates carry uncertainty.

A common pitfall is confusing notation, big N versus small n, sigma versus s. Interviewers often follow up with: *if your sample is huge, do you still need inference?* Yes, because a huge sample can still be unrepresentative. Selection bias does not disappear with size — it just gets more confident about the wrong answer.

Q32. Explain the difference between descriptive and inferential statistics with examples.

Descriptive statistics summarize the data you already have, with no claim beyond it. Examples are last month's average order value, distribution of customer ages, or top 10 products by revenue. The tools are mean, median, standard deviation, percentiles, histograms, and boxplots.

Inferential statistics use a sample to draw conclusions about a larger population, with uncertainty quantified. Examples are an A/B test concluding the new checkout flow lifts conversion by 2%, or a confidence interval for delivery time. Tools include hypothesis tests, confidence intervals, and regression coefficients with p-values.

```
df['order_value'].describe()           # descriptive
from scipy import stats
stats.ttest_ind(group_a, group_b)      # inferential
```

A common interview follow-up is: *is a dashboard descriptive or inferential?* Usually descriptive, it shows what happened. The moment you say *this difference is significant* or *users will behave like this*, you have moved into inferential territory.

Q33. When would you use mean vs median vs mode?

Mean is the average. Use it when data is roughly symmetric and free of extreme outliers, like heights or exam scores. **Median** is the middle value, best when data is skewed or has outliers, like income, house prices, or salary. One billionaire ruins the mean but barely moves the median. **Mode** is the most frequent value, used for categorical data like the most common payment method, or for discrete counts.

Quick example, salaries of 5 employees: 3L, 4L, 5L, 6L, 100L. Mean is 23.6L which is misleading, median is 5L which is honest.

```
df['salary'].mean(), df['salary'].median(), df['city'].mode()[0]
```

When asked for *average salary*, clarify whether they want a typical employee experience (median) or total payroll divided by headcount (mean). For skewed business metrics like revenue per user or session duration, prefer median or report both side by side.

Q34. Give the intuition behind variance and standard deviation.

Both measure spread — how much values differ from the mean. **Variance** is the average of squared distances from the mean. Squaring stops positive and negative gaps from cancelling out and penalizes large deviations more. **Standard deviation** is the square root of variance, brought back to the original units like rupees, seconds, or kilograms, which is why we report it on dashboards instead of variance.

Low standard deviation means values cluster tightly around the mean, like consistent delivery times. High standard deviation means scattered values, like volatile stock prices.

Consider two cities with the same mean delivery time of 30 minutes. City A is 30 ± 5 minutes, City B is 30 ± 20 minutes. Same average, very different customer experience, and B clearly needs investigation.

Standard deviation is sensitive to outliers. For a robust spread, interviewers may ask about IQR ($Q3 - Q1$) or median absolute deviation.

Q35. What are skewness and kurtosis in plain English?

Skewness measures the asymmetry of a distribution. *Positive (right) skew* has a long tail on the right and mean greater than median, like income, transaction amounts, or session duration. *Negative skew* has a long tail on the left and mean less than median, like age at retirement. Zero skew means symmetric.

Kurtosis measures tailedness — how heavy the tails are compared to a normal distribution. *High kurtosis* means fat tails and more extreme outliers, like stock returns. *Low kurtosis* means thin tails and fewer outliers.

```
df['amount'].skew(), df['amount'].kurt()
```

Interviewers ask this because many models like linear regression and t-tests assume roughly normal residuals. Heavy skew or kurtosis hints at log-transform, winsorizing, or robust methods. Risk and fraud

teams care about kurtosis because rare extreme events drive losses. A useful rule of thumb is that absolute skew above 1 is highly skewed and worth a transform.

Q36. Why is the normal distribution so important in statistics?

The normal distribution shows up everywhere for a few reasons. First, the **Central Limit Theorem** says averages of many independent variables tend to be normal regardless of the original distribution. Second, many natural phenomena like heights, measurement errors, and exam scores are approximately normal. Third, it is mathematically convenient, fully described by mean and standard deviation, and the **68-95-99.7 rule** makes probabilities easy to reason about.

It is also a foundational assumption for many techniques: linear regression residuals, t-tests, ANOVA, control charts, and Z-score outlier detection.

```
from scipy import stats
stats.norm.cdf(1.96)    # ~0.975
```

The pitfall is assuming your data is normal without checking. Use a histogram, Q-Q plot, or Shapiro test. Real business data like revenue, latency, or counts is usually right-skewed. Most modern ML algorithms like trees and gradient boosting do not need normality – it matters mainly for classical inference and parametric tests.

Q37. Explain the Central Limit Theorem (CLT) in plain English.

In one line, **if you take many random samples from any population and compute the mean of each, those sample means form a roughly normal distribution, even if the original data is not normal.**

The conditions are: samples are independent, sample size is reasonably large (n at least 30 as a rule of thumb), and the population has finite variance.

Why it is useful – it justifies using normal-based tools like Z-tests, t-tests, and confidence intervals on sample averages even when raw data is skewed. It is the reason A/B test math works on conversion rates and average revenue.

Intuition: imagine rolling a die which is uniform. Roll it 30 times and take the average. Repeat 10,000 times. The histogram of those averages will look like a bell curve centered at 3.5.

The pitfall is that CLT applies to the **distribution of the sample mean**, not to individual data points. Candidates often say *by CLT my data becomes normal*, which is wrong. For very heavy-tailed data, n = 30 is not enough.

Q38. What does a p-value actually mean? How do you explain it to a non-technical stakeholder?

Technically, a p-value is the probability of seeing data as extreme as or more extreme than what you observed, **assuming the null hypothesis is true**.

In plain English for a stakeholder: *if there were truly no difference between the old and new checkout flow, what is the chance we would still see a result this big purely by luck?* That chance is the p-value. A small p-value means luck is an unlikely explanation, so we lean towards the new flow actually being better.

Usage: $p < 0.05$ typically means reject null and call the result statistically significant. $p > 0.05$ means not enough evidence — do not conclude there is no effect.

Common misconceptions interviewers test: p-value is **not** the probability the null is true, **not** the probability your result happened by chance, and a non-significant p-value does **not** prove the null. Statistical significance is also not business significance — a 0.01% lift can be significant with huge n but commercially meaningless. Always pair p-values with effect size and confidence intervals.

Q39. What are null and alternative hypotheses? Give a business example.

The **null hypothesis (H_0)** is the default *nothing is happening* claim. It states no effect, no difference, or status quo. The **alternative hypothesis (H_1)** is what you actually want to demonstrate, that there is an effect or difference.

E-commerce example, testing a new product widget on Flipkart. H_0 says the new widget has the same conversion rate as the old one. H_1 says the new widget has a different (or higher) conversion rate. You collect data, run a test, and either reject H_0 with evidence the widget works, or fail to reject H_0 because there is no evidence yet.

Key points: we never *accept* H_0 , we only *fail to reject* it. Absence of evidence is not evidence of absence. The null is what you are trying to disprove, set up as the boring claim. Use one-sided tests only if the opposite direction is impossible or irrelevant. Most product A/B tests are two-sided to avoid bias.

Follow-up: if your sample size is too small, you may fail to reject a real effect — called **low statistical power**.

Q40. What is the difference between Type 1 and Type 2 errors? Which is worse?

A **Type 1 error** (false positive) is rejecting H_0 when it is actually true — claiming there is an effect when there is none. Its probability is α , typically 0.05. A **Type 2 error** (false negative) is failing to reject H_0 when

H_1 is actually true — missing a real effect. Its probability is β , and **power** = $1 - \beta$.

Which is worse depends on context. In a medical drug trial, Type 1 is dangerous because you approve an ineffective or harmful drug, so set α very strict at 0.01. In fraud detection, Type 2 is costly because missing actual fraud loses money, so tune the threshold to catch more fraud even if false alarms rise. In A/B testing a UI change, Type 1 means shipping a useless feature and Type 2 means missing an improvement — both have business cost.

The tradeoff is that lowering α increases β unless you increase sample size. **Power analysis** is how you control Type 2, by choosing the right n upfront.

Q41. Explain confidence intervals intuitively. What does a 95% CI really mean?

A confidence interval is a range of plausible values for a population parameter, computed from your sample. The correct interpretation of a 95% CI is: **if we repeated this sampling and CI procedure many times, about 95% of those intervals would contain the true population value.**

The wrong but common interpretation is *there is a 95% probability the true mean lies in this specific interval*. That is Bayesian credible interval language, not frequentist.

In practice, a CI tells you both the point estimate and the uncertainty. A CI of average order value [450, 470] is far more informative than just 460. A narrow CI means a precise estimate from large n or low variance, a wide CI means uncertain. **If a CI for a difference includes zero, the effect is not statistically significant** at that level.

```
from scipy import stats
import numpy as np
mean = np.mean(x); se = stats.sem(x)
ci = stats.t.interval(0.95, len(x)-1, mean, se)
```

Always prefer reporting CIs alongside p-values — they communicate effect size and precision together.

Q42. What is the difference between correlation and causation?

Correlation means two variables move together, measured by coefficients like Pearson r . It says nothing about cause. **Causation** means changing one variable directly changes the other.

Classic examples: ice-cream sales correlate with drowning deaths. The cause is summer temperature, a confounder. Cities with more hospitals also have more deaths, simply because bigger cities have more of both.

People confuse them due to **confounders** where a third variable drives both, **reverse causality** where Y causes X not the other way, **selection bias** where the sample filters the relationship, and plain coincidence with enough variables tested.

To establish causation, the gold standard is a randomized controlled experiment or A/B test, since randomization breaks confounding. Causal inference techniques like difference-in-differences, instrumental variables, and propensity score matching help when randomization is not possible.

Follow-up: if marketing spend correlates with revenue, can we say spend causes revenue? No — both might be driven by seasonality, or revenue might fund more marketing. Run a holdout experiment to find out.

Q43. When would you use Pearson correlation vs Spearman correlation?

Pearson correlation measures the **linear** relationship between two continuous variables. It is sensitive to outliers and assumes both variables are roughly normal for inference. **Spearman correlation** is Pearson applied to the **ranks** of the data. It measures a **monotonic** relationship — whether Y consistently increases or decreases with X, even non-linearly. It is robust to outliers and works with ordinal data.

Use Pearson for clean continuous roughly linear data, like height versus weight, or ad spend versus revenue after a log transform. Use Spearman when data has outliers, when the relationship is monotonic but curved like experience versus salary, or when one or both variables are ordinal like 1 to 5 ratings.

```
df[['x', 'y']].corr(method='pearson')
df[['x', 'y']].corr(method='spearman')
```

If Pearson is low but Spearman is high, the relationship is non-linear and Pearson is misleading you. Always plot a scatter first. Correlation only captures pairwise effects, not interactions or causation. Kendall's τ is a third option, more robust for small samples.

Q44. Walk me through how you would design and analyze an A/B test.

Step 1 — Define hypothesis and metric. H_0 is no change in conversion, H_1 is the new design changes conversion. Pick one primary metric like checkout conversion and track guardrail metrics like latency and refunds.

Step 2 — Power analysis. Decide $\alpha = 0.05$, power = 0.8, and a minimum detectable effect like 1%. Use a calculator to get required n per arm.

Step 3 – Randomization. Randomize at the user level – not session level – so the same user does not see both. Use stratification if traffic is uneven across geos or devices.

Step 4 – Run. Run the test for full business cycles, at least 1–2 weeks, to cover weekday and weekend effects. Do not peek and stop early – that inflates Type 1 error.

Step 5 – Analyze. Use a two-proportion z-test or t-test depending on the metric. Report lift, p-value, and confidence interval.

Key pitfalls: **sample ratio mismatch** from broken randomization, **novelty effect**, **multiple testing** if you check many metrics needing Bonferroni or FDR correction, and statistical significance not equalling business significance.

Q45. What are common sampling techniques and what biases should you watch out for?

Common sampling techniques: *Simple random sampling* gives every unit equal probability – easy but may under-represent small groups. *Stratified sampling* splits the population into strata like metro / tier-2 / tier-3 cities and samples proportionally, ensuring representation and lower variance. *Systematic sampling* picks every k-th item from an ordered list – simple but risky if the list has hidden periodicity. *Cluster sampling* picks whole clusters like pin codes, cheaper for fieldwork. *Convenience sampling* is whoever is easy to reach – fast but biased and best avoided for inference.

Common biases: *Selection bias* when certain groups are systematically excluded, like an online survey missing non-internet users. *Survivorship bias* when you study only winners, like analyzing only successful startups. *Non-response bias* when those who do not respond differ from those who do. *Sampling frame bias* when the list you sample from does not match the population. *Recall bias* when survey respondents misremember.

When given any dataset, ask **how it was collected**. Bias at collection time cannot be fixed by fancier modeling later.

Section 4 – Exploratory Data Analysis (Q46–Q60)

EDA questions test whether you can look at a dataset and form hypotheses before touching a model. interviewers love asking "You are given a CSV file – what would you do first?" – your answer should map cleanly to the workflow below.

Q46. What is Exploratory Data Analysis (EDA) and why is it important before modelling?

EDA is the process of summarising, visualising, and understanding a dataset before applying any ML model. The goal is to build intuition about the data, validate assumptions, and surface issues early. It detects data quality issues like missing values, duplicates, and wrong dtypes; reveals distributions, outliers, and skewness that affect model choice; and highlights relationships between features and the target. It also guides feature engineering decisions and catches red flags like target leakage or class imbalance.

A common interviewer follow-up is whether you have done EDA on a real dataset, so be ready with a concrete example: rows, columns, target, and two or three insights you found. The pitfall to avoid is jumping straight to `model.fit()` without EDA. **Garbage in, garbage out** – a five-minute EDA often saves hours of debugging poor model performance later, and gives you something concrete to discuss with the business team.

Q47. Walk me through the steps of a typical EDA workflow.

My standard flow:

1. **Load & inspect** – `df.shape` , `df.head()` , `df.info()` to check sizes and dtypes.
2. **Summary stats** – `df.describe(include='all')` , `df.isnull().sum()` for missing, `df.duplicated().sum()` for duplicates.
3. **Univariate** – histograms and boxplots for numeric, `value_counts` for categorical.
4. **Bivariate / multivariate** – correlation heatmap, scatter plots, `groupby` with the target.
5. **Outliers** – IQR or z-score methods.
6. **Target distribution & class balance** – for classification.
7. **Document** key findings for the team.

The interview tip I always give is to **tie EDA findings to actions**, for example: *I found 30% missing in column X, so I imputed with median because it was right-skewed.* That shows you connect EDA to modelling decisions, not just produce charts for their own sake.

Q48. Explain the difference between univariate, bivariate, and multivariate analysis.

These describe how many variables you analyse together.

- **Univariate** — one variable at a time. Goal is to understand its distribution, central tendency, and spread. Numeric: histograms, boxplots, mean, median, std. Categorical: bar charts, `value_counts`.
- **Bivariate** — two variables together to find relationships. Numeric vs numeric: scatterplots and correlation. Numeric vs categorical: boxplots, violins, or `groupby` mean. Categorical vs categorical: crosstab, stacked bar, chi-square.
- **Multivariate** — three or more variables: pairplot, correlation heatmap, PCA, faceted plots, or the `hue` parameter in seaborn.

The interview tip is to say you **start univariate, then look at each feature versus the target (bivariate), then check feature interactions (multivariate)**. That structure shows clear thinking and a repeatable approach.

Q49. How do you detect missing values and what are the common imputation strategies?

Detection is straightforward — `df.isnull().sum()` for counts, `df.isnull().mean()*100` for percentages, and a missingness heatmap. The strategy depends on context:

- **Drop rows** if missing is small, under 5%, and looks random.
- **Drop the column** if more than 50–60% is missing and it is not critical.
- **Mean / median** for numeric — median is safer for skewed data.
- **Mode** for categoricals.
- **Forward / backward fill** for ordered data.
- **KNN imputer** uses similar rows and works well when features are correlated.
- **Model-based** imputation like `IterativeImputer` or MICE predicts missing values using other columns — most accurate but costly.
- **Missing-indicator column** also helps because missingness itself can be informative.

The pitfall is **never imputing using the full dataset before train-test split** — that leaks test info into training. Fit imputer on train, transform on test.

Q50. How do you detect outliers, and when should you keep versus drop them?

Common detection methods include the **IQR rule** — values below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$. **Z-score** where $|z| > 3$ is typically flagged, assuming near-normal data. **Boxplots** show whiskers and points beyond visually. **Domain rules** also help — for example, age 200 or salary -5 are clearly invalid.

Code-wise, compute Q1 and Q3 with `df['x'].quantile([0.25, 0.75])`, get IQR, then filter rows outside the bounds.

Keep outliers when they are genuine rare events like fraud, churn, or high-value customers, when domain says they are valid, or when the model is robust like tree-based models. **Drop or cap them** when they are clearly data-entry errors, when they distort linear models like linear regression or k-means, or when there are very few in count. A good alternative is **winsorising** at the 1st and 99th percentile, or log-transforming instead of dropping. Always document the choice.

Q51. Which plots do you use to check the distribution of a variable, and what does a Q-Q plot tell you?

For distribution checks I use:

- **Histogram** for a quick view of shape, modes, and gaps — bin size matters.
- **KDE plot** gives a smooth density estimate and is good for comparing groups.
- **Boxplot** shows median, quartiles, and outliers compactly.
- **Q-Q plot** (quantile-quantile) plots data quantiles against theoretical normal quantiles. If points lie on the 45° line, the data is roughly normal. A curve at the ends means heavy or light tails, and an S-shape suggests skewness.

```
from scipy import stats
import matplotlib.pyplot as plt
stats.probplot(df['x'], dist='norm', plot=plt)
```

This matters because tests like the t-test, linear regression assumptions, and transforms like Box-Cox assume near-normality. The Q-Q plot is a quick visual check, much more informative than just a histogram. I also use `df.skew()` and `df.kurt()` as numeric companions to these plots.

Q52. How do you check skewness, and when do you apply log, sqrt, or Box-Cox transformations?

I check skewness using `df['col'].skew()` and a histogram with KDE. **Skew > 1 or < -1** is highly skewed, between 0.5 and 1 is moderately skewed, near zero is symmetric. Linear models, logistic regression, KNN, and k-means assume features are roughly symmetric and scaled, so transformations help. Right-skewed targets like income, price, or sales often benefit from log. Tree-based models like Random Forest and XGBoost generally do not need transforms because they split on thresholds.

For the choice of transform:

- `log / log1p` – strong right skew with positive values; `np.log1p` handles zeros gracefully.
- `sqrt` – mild right skew or count-like data.
- **Box-Cox** – finds an optimal lambda but requires strictly positive values.
- **Yeo-Johnson** – similar but allows zero and negative values.

The pitfall is never logging a column with negatives without shifting, and remember to **inverse-transform predictions** if you transformed the target.

Q53. How do you read a correlation heatmap, and what are its limitations?

A correlation heatmap shows pairwise Pearson correlation between numeric features:

```
import seaborn as sns
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', center=0)
```

Values range from -1 to $+1$. Close to $+1$ is a strong positive linear relationship, close to -1 is strong negative, near zero is little or no linear relationship. I look for high correlation between predictors (above 0.8), which signals **multicollinearity** and may need one to be dropped. I also look at the target row or column to find the strongest predictors.

The **limitations** are important to mention. It captures only **linear** relationships, missing U-shaped or non-linear patterns. It only works on numeric columns, so encode categoricals first or use Spearman or Cramer's V. It is sensitive to outliers and correlation does not imply causation. Alternatives worth mentioning are Spearman for monotonic non-linear, point-biserial for numeric vs binary, and mutual information for general dependencies.

Q54. What is a cardinality check for categorical features and why does it matter?

Cardinality is the number of unique categories in a column, checked with `df[col].nunique()` across object dtype columns. It matters because it drives encoding choices:

- **Low (2-10)** – safe for one-hot encoding.
- **Medium (10-50)** – OK with one-hot but creates many columns; consider target or frequency encoding.
- **High (100+)** – one-hot explodes the feature space; use target encoding, frequency encoding, hashing, or embeddings.

- **Very high (close to row count)** — usually an ID column. Drop it, since it brings high leakage risk if used naively.

Other red flags during the check are the same category written differently like `Mumbai` , `mumbai` , `MUMBAI` with trailing space, which need cleaning and standardising. Group rare categories under 1% into `Other` , and drop single-value columns that carry no information. I also look for typos and trailing spaces, common in real-world data from forms or Excel exports.

Q55. What are common target leakage red flags you watch for during EDA?

Target leakage means using information during training that would not be available at prediction time, leading to unrealistically good metrics that fail in production. Red flags during EDA include:

- A feature with near-perfect correlation to the target, like 0.99 — too good to be true.
- A feature whose name suggests it is computed *after* the event, like `total_payment_received` predicting `loan_default` .
- Timestamps where the feature time is after the target time.
- Aggregated features built using future data, like lifetime value when predicting first purchase.
- IDs or proxies that uniquely encode the target, or features added later by another team without context.

To investigate, check feature importance and dig into any single feature that dominates suspiciously. Ask the data owner *when* the column is populated. Train without that feature and see if performance drops to a reasonable level. **Leakage often only shows up in production** after deployment — catching it during EDA saves real money and credibility.

Q56. How do you detect class imbalance and why is it important to flag it early?

I detect imbalance using `df['target'].value_counts(normalize=True)*100` and a `countplot` . A 50:50 split is balanced, 80:20 is mild and usually fine, 95:5 or worse is severe and common in fraud, churn, or rare disease problems.

Flagging it during EDA matters because:

- **Accuracy becomes misleading** — a 99% majority dataset gives 99% accuracy by predicting only the majority class.
- You need to switch metrics to **precision, recall, F1, ROC-AUC, or PR-AUC**.

- You also decide on a handling strategy early – SMOTE, class weights, undersampling, oversampling, or threshold tuning.
- It also affects train-test split strategy – you must use stratified splits to preserve the ratio:
`train_test_split(X, y, stratify=y, test_size=0.2)`.

The interview tip I always share is to mention **PR-AUC as preferred over ROC-AUC for highly imbalanced problems** – interviewers love that nuance and it shows real-world experience.

Q57. What do automated EDA tools like ydata-profiling and Sweetviz offer, and what are their limits?

Automated EDA tools generate a quick HTML or notebook report covering most basic checks in one shot:

- **ydata-profiling** (formerly pandas-profiling) – stats, missing values, correlations, distributions, alerts for high cardinality, skew, or constant columns.
- **Sweetviz** – great for train vs test comparison and has built-in target analysis.
- **AutoViz** – auto-picks plots based on dtypes.
- **D-Tale** – interactive Excel-like UI on a DataFrame.

```
from ydata_profiling import ProfileReport
ProfileReport(df, title='EDA').to_file('eda.html')
```

They are useful for first-pass exploration on a new dataset, sharing a snapshot with non-technical stakeholders, and quick sanity checks before deeper manual EDA.

The **limits** are real – they are slow and memory-heavy on millions of rows, can miss domain-specific issues a human would catch, generic plots may not surface the right insight, and they can overwhelm with too many charts. Treat them as a starting point, **not** a substitute for thoughtful manual EDA.

Q58. How do you handle high-cardinality categorical features during EDA and preprocessing?

When a categorical column has hundreds or thousands of unique values, like pincode, product_id, or city, one-hot encoding explodes the feature space and slows training. During EDA, I first check `value_counts` and the long tail. Often the top 20 categories cover 80% of rows, so I group the rest under `Other`, which keeps signal and reduces noise.

- **Frequency encoding** replaces each category with its count – simple and works well with tree models.

- **Target encoding** replaces each category with the mean target value, but you must compute it on training folds only, otherwise it leaks.
- **Hashing trick** maps categories into a fixed number of buckets — useful when categories grow over time.
- **Embeddings** are the cleanest option for deep learning.

I also flag the column for the data owner if cardinality looks suspicious, like a free-text column meant to be a dropdown. Always document the chosen encoding because it affects model interpretability and downstream serving.

Q59. When do you choose a scatterplot vs a boxplot vs a violin plot?

Each plot answers a different question:

- **Scatterplot** — numeric vs numeric. Used when you want to see the relationship between two continuous variables, linearity, clusters, or outliers. `sns.scatterplot(x='age', y='income', data=df)`.
- **Boxplot** — numeric vs categorical, or a single numeric. Used to compare distributions across groups, showing median, IQR, and outliers compactly. `sns.boxplot(x='city', y='salary', data=df)`.
- **Violin plot** — like a boxplot but adds the density/shape of the distribution. Used when the shape matters, e.g. bimodality or non-normal distributions that a boxplot would hide. `sns.violinplot(x='gender', y='spend', data=df)`.

The quick rule of thumb is: two continuous → scatter, continuous by group with a summary needed → box, continuous by group where shape matters → violin.

The pitfall is that **boxplots hide bimodality** — two very different distributions can have identical boxplots. Violin or histogram-by-group reveals it.

Q60. What are the key insights from EDA you would communicate to business stakeholders?

Stakeholders care about *what the data tells us about the business problem*, not how you computed correlations. The structure I follow:

1. **Data quality summary** — total rows, percent missing, any duplicates or invalid entries. Sets expectations for trust.
2. **Target overview** — class distribution or target distribution. Example: only 4% of customers churned, so we have an imbalance problem.

3. **Top drivers** – three to five features that look most related to the target with simple charts.
Example: churn is 3× higher for customers on month-to-month contracts.
4. **Segments / patterns** – any clusters or cohorts that stand out by region, age group, or product.
5. **Outliers and anomalies** – flag suspicious records and ask if they are valid.
6. **Risks** – missing data, possible leakage, imbalance, sample bias.
7. **Next steps** – what features to engineer or what data to collect more of.

Use plain language, one chart per insight, and tie everything back to a business KPI.

Section 5 – Feature Engineering (Q61–Q75)

Feature engineering is where good candidates separate from great ones. interviewers (especially in BFSI, fintech, and e-commerce) probe deeply on encoding choices, scaling, and leakage prevention. Always mention `Pipeline` and `ColumnTransformer`.

Q61. What is feature engineering and why is it often said to matter more than model choice?

Feature engineering is the process of turning raw data into inputs that better expose the underlying problem to a model. It covers cleaning, encoding, scaling, creating new variables, and selecting useful ones.

It matters more than the model because algorithms can only learn patterns that are actually present in the features. A linear model on great features usually beats XGBoost on raw garbage. The industry saying is, "**better data beats fancier algorithms.**" Good FE reduces variance, captures non-linearity, and bakes in domain knowledge that no model can invent on its own.

A common pitfall is jumping straight to hyperparameter tuning before fixing missing values or leakage. Interviewers like candidates who say they would **start with EDA and FE before model selection**. Worth noting: deep learning learns features automatically for images and text, but for tabular data – which dominates analytics work – manual FE still wins on Kaggle and in production.

Q62. Explain label encoding vs one-hot encoding vs ordinal encoding. When do you use each?

All three convert categories to numbers, but the assumption behind each is different.

- **Label encoding** assigns 0, 1, 2 arbitrarily. Use it for the **target variable** in classification, or as input to **tree-based models** where order doesn't mislead the splits.

- **Ordinal encoding** uses the same mechanic but the order is meaningful, like Low/Medium/High, or education levels. You define the mapping yourself.
- **One-hot encoding** creates a binary column per category. Use it for nominal features with **linear or distance-based models** like Logistic Regression, KNN, or SVM.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

Pitfalls: label encoding on nominal features with a linear model implies a fake ranking. One-hot blows up dimensionality on high-cardinality columns, so frequency or target encoding is better there. Always set `handle_unknown='ignore'` so unseen test categories don't crash the pipeline.

Q63. Why treat missingness as a feature, and how do you do it?

Missingness is rarely random. The reason a value is missing often carries signal. A customer not filling income may indicate informal employment, which itself is predictive.

The common trick is a **missingness indicator** — add a binary `col_is_missing` column before imputing.

```
df['income_missing'] = df['income'].isna().astype(int)
df['income'] = df['income'].fillna(df['income'].median())
```

For numeric columns, **median** is robust to outliers. For categorical, use the **mode** or a new `'Unknown'` bucket. Advanced options include KNN imputer or iterative imputer when missingness is structured.

Pitfalls: computing the median on the full dataset before splitting **leaks test info**, so always fit the imputer on train only. Dropping rows with NaN can bias the sample if missingness correlates with the target. For XGBoost or LightGBM you can pass NaNs directly because they handle missing splits natively, so imputation may not even be needed.

Q64. Compare StandardScaler, MinMaxScaler, and RobustScaler. Which models need scaling?

All three rescale numeric features but differ in approach:

- **StandardScaler** — $(x - \text{mean}) / \text{std}$, centering at zero with unit variance. Default choice.

- **MinMaxScaler** — $(x - \min) / (\max - \min)$, squashing values to $[0,1]$. Useful for neural networks or image pixels.
- **RobustScaler** — uses median and IQR, so it resists outliers and is good for heavy-tailed data.

Models that need scaling: distance-based ones like KNN, K-Means, SVM, and PCA; gradient-based ones like Linear and Logistic Regression and neural networks; regularized models like Ridge and Lasso where the penalty depends on coefficient magnitude.

Models that don't need scaling: tree-based — Decision Trees, Random Forest, XGBoost, LightGBM. Their splits are scale-invariant.

Pitfall: fitting the scaler on the entire dataset before splitting causes leakage. Always do `scaler.fit(X_train)` then `transform(X_test)` . Inside cross-validation, wrap it in a `Pipeline` so scaling refits per fold.

Q65. When and why do you apply log, sqrt, or power transforms?

These transforms reduce skewness and stabilize variance for right-skewed columns like income, prices, transaction amounts, or counts.

- **Log transform**, usually `np.log1p(x)` to handle zeros, is best for highly right-skewed data.
- **Sqrt** is milder and works well for count data.
- **Box-Cox** or **Yeo-Johnson** via `PowerTransformer` auto-selects the best power. Yeo-Johnson handles negatives and zeros.

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
X_t = pt.fit_transform(X)
```

Why it helps: linear models assume residuals are roughly normal, transforms pull outliers closer reducing their leverage, and they often improve R^2 noticeably in regression.

Pitfall: don't blindly transform every column. Check skew with `df.skew()` first and transform only when $|\text{skew}| > 1$. **If you transform the target, remember to invert predictions** before reporting metrics, otherwise your RMSE will be in log-space and meaningless to the business.

Q66. What is binning and what's the difference between equal-width and equal-frequency binning?

Binning converts a continuous variable into categorical buckets. It's useful when relationships are non-linear or when the business needs interpretable groups like age bands or salary slabs.

- **Equal-width binning** splits the range into k intervals of equal size using `pd.cut(df['age'], bins=5)`. Simple but sensitive to outliers – one extreme value stretches the range and most data lands in one bin.
- **Equal-frequency (quantile) binning** puts roughly the same number of points in each bin using `pd.qcut(df['age'], q=5)`. Robust to outliers and works well on skewed data.

When to bin: linear models that need non-linear behaviour, credit risk buckets in BFSI and fintech interviews, and reducing noise in jittery continuous features.

Pitfalls: binning loses information, and trees can learn splits on their own so it's rarely needed for them.

Compute bin edges only on the training set, otherwise quantile boundaries leak test data into the model.

Q67. How do you extract useful features from a date column?

Even outside forecasting problems, dates carry behavioural signal: weekend vs weekday spending, salary-day spikes, festival months. The standard approach is to break the date into components.

```
df['date'] = pd.to_datetime(df['date'])
df['day'] = df['date'].dt.day
df['month'] = df['date'].dt.month
df['dayofweek'] = df['date'].dt.dayofweek
df['is_weekend'] = df['dayofweek'].isin([5,6]).astype(int)
df['quarter'] = df['date'].dt.quarter
df['is_month_end'] = df['date'].dt.is_month_end.astype(int)
```

For **cyclical fields** like month or day of week, use `sin(2 $\pi$ ·x/period)` and `cos(...)` so December and January are close in feature space. **Domain features** matter too: `is_salary_day` for the 1st–7th of the month, or `is_festival_month` for Diwali and Eid.

Pitfall: feeding a raw datetime as a number is meaningless to most models. Always extract components, and drop the original date column afterwards so trees don't overfit on exact timestamps.

Q68. What are interaction features and when should you create them?

Interaction features capture combined effects of two or more variables, where the impact of feature A on the target depends on feature B.

Common examples: `price_per_sqft = price / area` for real estate,

`income_per_dependent = income / num_dependents` for credit scoring, `clicks * impressions` for ad CTR models.

```
df['bmi'] = df['weight'] / (df['height']**2)
df['city_x_product'] = df['city'].astype(str) + '_' + df['product'].astype(str)
```

Create them when you're using a linear or logistic model that can't learn interactions on its own, when domain knowledge suggests a ratio or product, or when EDA shows the target depends on combined behaviour.

Tree-based models can learn interactions implicitly through splits, but explicit ones still help shallow trees and small datasets.

Pitfall: don't blindly cross every pair. The feature space explodes and overfitting risk rises. Be guided by domain knowledge or feature importance from a quick baseline model, and validate that the interaction actually improves cross-validation score.

Q69. How do you handle high-cardinality categorical features?

Columns like `user_id`, `pincode`, or `product_sku` may have thousands of levels, which makes one-hot encoding impractical.

Frequency / count encoding replaces each category with how often it appears, capturing a popularity signal:

```
freq = df['pincode'].value_counts()
df['pincode_freq'] = df['pincode'].map(freq)
```

Target encoding with smoothing replaces categories with the mean target, regularised so rare levels regress towards the global mean. The **hashing trick** maps categories to a fixed number of buckets, giving constant memory and graceful handling of unseen values, with a few collisions. **Embeddings** learn dense vectors via a neural net and are common in deep learning pipelines. **Grouping** merges rare categories below a frequency threshold into `'Other'`.

Pitfall: always fit the encoder **on train only** and apply to test, otherwise leakage is guaranteed. For freshers, frequency encoding plus an `'Other'` bucket is a safe, defensible default in interviews – and target encoding inside CV folds is the next level up.

Q70. What is target encoding and what is the leakage risk?

Target encoding replaces each category with a statistic of the target, usually the mean, computed for that category. It's useful for high-cardinality features like PIN code, product_id, or city.

```
means = train.groupby('city')['target'].mean()
train['city_te'] = train['city'].map(means)
```

The **leakage risk** is real — if you compute the means using the same rows you train on, the model effectively sees the target. Validation accuracy looks great and production fails.

Fixes are:

- Compute means **inside CV folds** (out-of-fold encoding).
- Add **smoothing** like $(n \cdot \text{mean_cat} + m \cdot \text{global_mean}) / (n + m)$ so rare categories regress towards the global mean.
- Optionally add small Gaussian noise to break memorisation.

A likely follow-up is, *why is it better than one-hot for high cardinality?* Because it compresses category info into one numeric feature without exploding dimensions, trees split on it cleanly, and it works well for linear models too once smoothing is applied.

Q71. What are polynomial features? When are they useful or harmful?

Polynomial features generate higher-order terms and interactions of existing numeric features. For features $[a, b]$ with degree 2, you get $[a, b, a^2, ab, b^2]$.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

Useful when the underlying relationship is curved (like price vs area), when you're using a linear model and need non-linearity without switching to trees, and when you have few features but enough data.

Harmful when there are many features, because the combinatorial explosion is brutal. Ten features at degree 3 produce hundreds of columns. Multicollinearity also skyrockets, so always pair with **Ridge or Lasso**, and they easily overfit small datasets.

Pitfall: always **scale before polynomial expansion**, otherwise large values dominate the squared and cubed terms. For tree-based models, polynomial features are usually unnecessary because splits already handle non-linearity well.

Q72. Explain Bag of Words and TF-IDF at a high level.

Both convert text into numeric vectors so ML models can consume it.

Bag of Words builds a vocabulary of unique words, then represents each document as a vector of word counts. It ignores order and grammar.

```
from sklearn.feature_extraction.text import CountVectorizer
bow = CountVectorizer(ngram_range=(1,2), max_features=5000)
```

TF-IDF – term frequency times inverse document frequency – down-weights words that appear in many documents like “the” or “is”, and up-weights rare, distinctive words. The formula is roughly $TF \times \log(N / df)$.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(stop_words='english', ngram_range=(1,2))
```

TF-IDF is usually preferred over plain BoW because it filters common-word noise. Pitfalls: vocabulary explosion is real, so cap with `max_features` or `min_df`. Fit only on **train data** to avoid leakage. For semantic understanding like sarcasm or context, embeddings or transformers are better – but BoW and TF-IDF still win on small text classification tasks like ticket routing.

Q73. How do you handle outliers as a feature engineering step?

Outliers can distort means, scaling, and linear models. First decide whether they're errors or genuine signal. **A fraud transaction is an outlier you want to detect, not remove.**

Detection methods include the IQR rule (outside $Q1 - 1.5 \cdot IQR$ to $Q3 + 1.5 \cdot IQR$), z-score above 3 assuming normality, and **Isolation Forest** or DBSCAN for multivariate cases.

Treatment options:

```
# Capping / winsorisation
lower, upper = df['amt'].quantile([0.01, 0.99])
df['amt'] = df['amt'].clip(lower, upper)
```

Log or power transforms reduce influence without dropping data. Robust models like trees or `RobustScaler` help. You can also add an `is_outlier` binary flag.

Pitfalls: removing outliers blindly throws away rare-but-real events like fraud or churn, which is exactly what you're often trying to predict. Always compute thresholds **on training data only**. Linear models suffer most from outliers, while tree-based models are largely robust.

Q74. Briefly explain filter, wrapper, and embedded feature selection methods.

Feature selection picks a useful subset to reduce overfitting, training time, and noise.

- **Filter methods** use statistics independent of the model, like correlation with target, Chi-square, ANOVA F-test, or Mutual Information. Fast and scalable but ignore feature interactions.

```
from sklearn.feature_selection import SelectKBest, f_classif
SelectKBest(f_classif, k=20).fit_transform(X, y)
```

- **Wrapper methods** train models on different subsets and pick the best, like Recursive Feature Elimination (RFE), or forward and backward selection. Accurate but expensive – impractical for thousands of features.
- **Embedded methods** do selection during training. **L1 (Lasso)** regression sets coefficients to zero, and **tree models give feature importances** natively. This is the best balance of speed and quality and is most common in practice.

Pitfall: doing selection on the full dataset before cross-validation **leaks information**. Always wrap selection inside a sklearn `Pipeline` so it runs per fold, otherwise your reported scores are optimistic.

Q75. What is data leakage in feature engineering, and why must FE happen inside CV folds?

Data leakage happens when information from the test set sneaks into training, inflating validation scores while production performance crashes.

Common FE leaks:

- Computing mean, median, or scaler stats on the full dataset before splitting.
- Target encoding using all rows.
- Feature selection that peeks at the test target.
- Imputing with the global mean.

The fix is sklearn's `Pipeline`. It ensures every transformation is fit on train only, then applied to validation or test:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('sc', StandardScaler()),
    ('clf', LogisticRegression())
])
cross_val_score(pipe, X, y, cv=5)
```

Why **inside CV folds**: each fold should mimic production where the model has never seen the validation rows. If you encode or scale before `cross_val_score`, every fold's "unseen" data was already used to fit the transformers. This is the **single biggest mistake interviewers probe for** – so mention `Pipeline` and `ColumnTransformer` confidently.

Section 6 – Machine Learning (Q76–Q100)

ML questions test breadth (algorithms), depth (when each one fits), and judgement (debugging a model that works in dev but fails in prod). Expect heavy focus on overfitting, metrics, class imbalance, and `sklearn` mechanics.

Q76. What is the difference between supervised, unsupervised, and reinforcement learning?

These are the three broad ML paradigms, separated by what feedback the model gets.

- **Supervised learning** – we have labelled data (X, y). The model learns a mapping from inputs to known outputs. Examples: spam detection, house price prediction. Common algorithms are Logistic Regression, Random Forest, XGBoost.
- **Unsupervised learning** – only X, no labels. The model finds structure on its own. Examples: customer segmentation with K-Means, dimensionality reduction with PCA, anomaly detection.
- **Reinforcement learning** – an agent interacts with an environment, takes actions, and learns from rewards and penalties over time. Examples: game-playing bots, robotics, dynamic pricing.

In service-based interviews, around 90% of work is supervised. A common pitfall is confusing clustering (unsupervised) with classification (supervised). Expect a follow-up like, "*Is fraud detection supervised or unsupervised?*" Usually supervised when fraud labels exist, but anomaly detection is used when labels are scarce or noisy.

Q77. Classification vs regression – how do you decide which one a problem is?

Look at the **target** variable, not the features.

- **Classification** – target is categorical or discrete. Output is a class label or probability. Binary examples are churn yes/no and fraud detection. Multi-class examples are product category or sentiment.
- **Regression** – target is continuous numeric. Output is a real number. Examples include salary prediction, demand forecasting, and customer lifetime value.

Quick test: can the target meaningfully be averaged? If yes (price, age) it's regression. If no (cat, dog) it's classification.

Pitfalls: predicting *age* can be either, since bucketed groups like child/adult/senior make it classification. Predicting *probability of default* is classification, even though the output is between 0 and 1. Ordinal targets like rating 1–5 can be tackled as classification, regression, or ordinal regression – interviewers love this nuance.

Metrics also differ: accuracy, F1, ROC-AUC for classification; RMSE, MAE, R^2 for regression.

Q78. Why do we split data into train, validation, and test sets?

To get an honest estimate of how the model performs on **unseen** data and to avoid fooling ourselves.

- **Train set** (~60–70%) – the model learns parameters here.
- **Validation set** (~15–20%) – used for hyperparameter tuning, model selection, and early stopping.
- **Test set** (~15–20%) – touched only **once**, at the very end, for the final unbiased number.

If we tune on the test set, we leak information and the score becomes optimistic.

```
from sklearn.model_selection import train_test_split
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.3, random_s
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5,
```

Pitfalls: forgetting `stratify` for imbalanced data, splitting **after** preprocessing (which causes leakage), and using the test set for tuning. Always **split first, then fit transforms on train only**.

Q79. Explain k-fold cross-validation and when you would use it.

K-fold CV gives a more reliable performance estimate than a single train/val split, especially on small datasets.

How it works: split training data into k equal folds (commonly k = 5 or 10). Train on k-1 folds, validate on the held-out fold. Repeat k times so every fold serves as validation once. Average the k scores.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print(scores.mean(), scores.std())
```

Why use it: a more stable metric reduces variance from a lucky/unlucky split, uses all data for both training and validation, and works well for hyperparameter tuning via `GridSearchCV`.

Pitfalls: it's costly because it trains the model k times. For imbalanced data, use `StratifiedKFold`. For grouped data like multiple rows per customer, use `GroupKFold` to prevent leakage. Always keep a final hold-out test set even when using CV — CV only replaces the train/val split.

Q80. What is stratified k-fold and why is it preferred for classification?

Stratified k-fold preserves the **class proportion** in every fold, mirroring the overall dataset distribution.

Example: a churn dataset with 90% non-churn and 10% churn. With plain `KFold`, one fold could randomly end up with 2% or 18% churn, making metrics unstable. `StratifiedKFold` guarantees each fold has roughly 10% churn.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='f1')
```

When to use: **any** classification, especially imbalanced problems like fraud, default, or rare disease. Also for small datasets where one fold may otherwise miss a class entirely.

Pitfalls: don't use `StratifiedKFold` for regression — use `KFold` or stratify on a binned target. Sklearn's `cross_val_score` actually applies `StratifiedKFold` by default for classifiers, but be explicit in interviews. For multi-label problems, use `MultilabelStratifiedKFold` from `iterstrat`.

Q81. Explain the bias-variance tradeoff in plain English.

Every model's error breaks down into bias, variance, and irreducible noise.

- **Bias** — error from the model being too simple to capture the pattern. High bias means **underfitting**. Example: fitting a straight line to clearly curved data.
- **Variance** — error from the model being too sensitive to the training data. High variance means **overfitting**. Example: a deep decision tree memorising noise.
- **Irreducible error** — noise we cannot model away.

The tradeoff: making the model more flexible reduces bias but increases variance, and vice versa. The sweet spot minimises total error.

Practical signals: **high train and test error** means high bias — use a more complex model or add features. **Low train but high test error** means high variance — regularise, add data, or simplify. Both low and close means a good fit.

Pitfall: candidates often only mention overfitting. Mention both directions and tie them to learning curves and model complexity for full marks.

Q82. What is overfitting vs underfitting, and how do you fix each?

Underfitting: the model is too simple and performs poorly on both train and test. It hasn't learned the pattern.

Fixes: use a more complex model (move from linear to tree-based), add or engineer better features, reduce regularisation, or train longer for neural nets and boosting.

Overfitting: the model performs great on train but poorly on test — it memorised noise.

Fixes: get more training data, apply regularisation (L1/L2, dropout), reduce complexity (shallower tree, fewer features), use cross-validation for honest evaluation, apply early stopping in XGBoost or NNs, use bagging or ensembling, and remove noisy or leaky features through selection.

Quick diagnostic: plot a learning curve. A big train-vs-val gap means overfitting. Both curves stuck at high error means underfitting.

Pitfall: don't only say "add more data" — interviewers want to hear regularisation and CV. Also clarify whether you mean training data quantity or feature richness.

Q83. Explain L1 vs L2 regularization in plain English.

Regularization adds a penalty on model weights to discourage overfitting — it shrinks coefficients toward zero.

- **L2 (Ridge)** – penalty is the sum of squared weights. Shrinks all weights smoothly toward zero but rarely makes them exactly zero. Good when many features each contribute a little.
- **L1 (Lasso)** – penalty is the sum of absolute weights. Can drive some weights to **exactly zero**, effectively performing feature selection. Good when you suspect only a few features matter.
- **ElasticNet** – a mix of both. Useful when features are correlated.

```
from sklearn.linear_model import Lasso, Ridge, ElasticNet
Lasso(alpha=0.1).fit(X, y)
Ridge(alpha=1.0).fit(X, y)
```

When to use: L1 if you want a sparse, interpretable model; L2 if all features are believed relevant.

Pitfalls: always **scale features** before applying regularisation – the penalty is unfair otherwise. Tune `alpha` via cross-validation. In sklearn's logistic regression, `C = 1/alpha` – note the inversion.

Q84. Explain gradient descent intuitively (no math).

Imagine standing on a foggy hill and wanting to reach the bottom. You can't see far, so you feel the slope under your feet and take a small step in the steepest downward direction. Repeat. **That's gradient descent.**

The hill is the loss function. The slope is the gradient (direction of steepest *increase*, so we go opposite). The step size is the **learning rate**. Model parameters update iteratively until the loss stops decreasing.

Variants:

- **Batch GD** uses the entire dataset per step – accurate but slow.
- **Stochastic GD** uses one sample per step – noisy but fast and can escape shallow minima.
- **Mini-batch GD** uses 32–128 samples – best of both, used in practice.

Pitfalls: a learning rate too high means overshooting and diverging loss. Too low means it trains forever. **Unscaled features cause a zigzag path** and slow convergence – always standardise. Saddle points are more of a problem than local minima in practice. Adam and RMSProp are common adaptive optimisers.

Q85. What are the assumptions of Linear Regression?

Linear regression has five classical assumptions:

1. **Linearity** – relationship between X and y is linear. Check via residual plots.
2. **Independence** – observations are independent (no autocorrelation). Check Durbin-Watson.

3. **Normality of residuals** – errors should be normally distributed. This matters mainly for inference and confidence intervals. Use a Q-Q plot.
4. **Homoscedasticity** – residuals have constant variance across X. A funnel-shaped residual plot signals heteroscedasticity.
5. **No multicollinearity** – predictors shouldn't be highly correlated. Check VIF; over 5 or 10 is a red flag.

Also desirable: no influential outliers, checked with Cook's distance.

Pitfalls: assumption violations hurt **coefficient interpretation** more than predictive accuracy. Fixes include log-transforming skewed targets, dropping or combining collinear features, using Ridge for multicollinearity, and robust regression for outliers. For pure prediction, **tree-based models don't need any of these assumptions** – a great follow-up to add.

Q86. Why is Logistic Regression used for classification despite being called "regression"?

Logistic regression models the **probability** of belonging to a class, which is a continuous output between 0 and 1. We then threshold (default 0.5) to get a class label, making it a classifier.

It fits a linear combination $z = w \cdot x + b$ and passes it through the **sigmoid** function: $\text{sigmoid}(z) = 1 / (1 + e^{-z})$. The sigmoid squashes any real number into (0,1) – perfect for probabilities. "Regression" stays in the name because we're regressing the **log-odds**.

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(C=1.0, class_weight='balanced').fit(X, y)
proba = clf.predict_proba(X_test)[: , 1]
```

Strengths: interpretable coefficients (odds ratios), well-calibrated probabilities, and a great baseline for binary problems.

Pitfalls: assumes a linear decision boundary in feature space, sensitive to multicollinearity, and needs feature scaling when regularised. **The threshold of 0.5 is not sacred** – tune it for your business metric.

Q87. What's the difference between Gini and Entropy in decision trees?

Both measure **node impurity** – how mixed the classes are at a tree node. The split that reduces impurity the most is chosen.

- **Gini impurity** – probability that a randomly picked sample is misclassified. Range 0 (pure) to 0.5 (binary worst case).

- **Entropy** – from information theory; measures disorder. Range 0 to 1 (binary worst case).

In practice both produce **very similar trees**. Gini is slightly faster (no log), so it's the sklearn default.

```
DecisionTreeClassifier(criterion='gini')
DecisionTreeClassifier(criterion='entropy')
```

Decision tree **pros**: interpretable, no scaling needed, handles mixed data, captures non-linearities and interactions.

Cons: highly prone to overfitting, unstable since small data changes produce very different trees, and greedy splits may miss the global optimum.

Pitfall: a candidate saying "*entropy is more accurate*" – usually not. The fix for instability is **ensembling** (Random Forest, boosting), not picking a different criterion.

Q88. How does Random Forest work and why is it usually better than a single decision tree?

Random Forest is an ensemble of decision trees built using **bagging** (Bootstrap Aggregation) plus feature randomness.

1. Draw N bootstrap samples (sampling with replacement).
2. Train one decision tree per sample, but at each split consider only a random subset of features (`sqrt(p)` for classification).
3. Predict by majority vote for classification or average for regression.

Why better than a single tree: single trees have low bias but very high variance. Averaging many de-correlated trees dramatically **reduces variance**. Feature randomness ensures trees don't all latch onto the same dominant feature. The built-in **OOB (out-of-bag) score** gives free validation, and feature importance is provided.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=300, max_features='sqrt', n_jobs
```

Pitfalls: less interpretable than a single tree, slower to predict, and can still overfit on noisy data with too many deep trees. Tune `max_depth`, `min_samples_leaf`, and `n_estimators`.

Q89. Give an overview of boosting and how it differs from bagging.

Boosting builds an ensemble **sequentially**, where each new model focuses on the errors of the previous ones.

1. Start with a weak learner, often a shallow tree of depth 3-6.
2. Compute residuals or errors.
3. Train the next learner to predict those errors.
4. Add it to the ensemble with a small learning rate.
5. Repeat for N iterations. Final prediction is the weighted sum of all learners.

Bagging vs boosting: bagging trains in **parallel** to reduce **variance** using deep, independent trees. Boosting trains **sequentially** to reduce **bias and variance** using shallow, dependent trees. Boosting carries higher overfitting risk and needs early stopping.

Common algorithms: **AdaBoost** re-weights misclassified samples. **Gradient Boosting** fits residuals. **XGBoost, LightGBM, and CatBoost** are optimised gradient boosting libraries.

Pitfalls: boosting is **sensitive to noisy labels** because it keeps trying to fit them. It needs tuning of `learning_rate`, `n_estimators`, and `max_depth`. Always use early stopping on a validation set.

Q90. Why are XGBoost and LightGBM so popular in practice and interviews?

Both are highly optimised gradient boosting libraries that consistently win Kaggle competitions and real-world tabular problems.

Common strengths: handle missing values natively, built-in L1/L2 regularisation plus tree pruning, support categorical features (LightGBM directly, XGBoost via `enable_categorical`), parallelised, GPU-friendly, and fast on large data. Both offer early stopping and feature importance.

- **XGBoost** uses **level-wise** tree growth – balanced trees, very stable.
- **LightGBM** uses **leaf-wise** growth – grows the leaf with maximum loss reduction. Faster and more accurate on large datasets but can overfit on small data unless you cap `num_leaves` or `max_depth`.
- **CatBoost** has the best handling of categorical features out of the box.

```
import lightgbm as lgb
model = lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05, num_leaves=31)
model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], callbacks=[lgb.early_stopping])
```

Pitfalls: don't blindly use defaults; tune `learning_rate`, `max_depth`, `min_child_samples`, and `reg_lambda`. Always use early stopping and stratified validation.

Q91. How does KNN work and what is the curse of dimensionality?

KNN (K-Nearest Neighbours) is a **lazy, instance-based** algorithm. There's no training phase — it just stores the data.

To predict for a new point: compute distance (Euclidean, Manhattan, cosine) to all training points, pick the k nearest, then majority vote for classification or average for regression.

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5, weights='distance').fit(X_tr, y)
```

Curse of dimensionality: as the number of features grows, distances between points become similar — every point looks roughly equidistant. KNN's notion of "*nearness*" breaks down, accuracy collapses, and computation explodes.

Practical implications: always **scale features** (StandardScaler or MinMaxScaler) before KNN. Apply dimensionality reduction (PCA, feature selection) when p is large. Choose k via cross-validation; small k overfits, large k underfits. Use odd k for binary problems to avoid ties. KNN is slow at prediction time on big datasets — use approximate NN libraries like FAISS or Annoy in production.

Q92. Explain the intuition behind Naive Bayes and when to use it.

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "**naive**" assumption that all features are conditionally independent given the class.

Intuition: given an email, what's $P(\text{spam} \mid \text{words})$? Bayes flips this to $P(\text{words} \mid \text{spam}) \times P(\text{spam})$, which is easy to estimate by counting word frequencies in spam emails. The independence assumption is almost always wrong, but the model still works surprisingly well, especially on text.

Variants:

- **GaussianNB** — continuous features assumed normal.
- **MultinomialNB** — word counts (text classification).
- **BernoulliNB** — binary features.

```
from sklearn.naive_bayes import MultinomialNB
nb = MultinomialNB().fit(X_tfidf, y)
```

When to use: text classification (spam, sentiment, topic), a quick baseline on high-dimensional sparse data, and small training sets since it needs little data to estimate parameters.

Pitfalls: probabilities are often poorly calibrated, struggles with correlated features, and the **zero-frequency problem** is solved via Laplace smoothing using `alpha` .

Q93. Explain the intuition behind SVM and the concept of margin.

SVM (Support Vector Machine) finds the decision boundary that best separates the classes with the **maximum margin**.

Margin: distance between the boundary and the closest data points on either side. **Support vectors** are those closest points – they alone define the boundary; others don't matter. A wider margin generally generalises better.

Soft margin: real data isn't perfectly separable, so we allow some misclassifications controlled by hyperparameter `C` . Large C means less tolerance, narrow margin, may overfit. Small C means more tolerance, wide margin, may underfit.

Kernel trick: when data isn't linearly separable, kernels like **RBF** or **polynomial** implicitly project data into higher dimensions where it becomes separable, without computing the projection explicitly.

```
from sklearn.svm import SVC
SVC(C=1.0, kernel='rbf', gamma='scale').fit(X_tr, y_tr)
```

Pitfalls: scale features first, slow on large datasets above 100k rows, no native probabilities (use `probability=True` or Platt scaling), and tune `C` and `gamma` together via grid search.

Q94. How does K-Means work and how do you choose the right k?

K-Means is an unsupervised clustering algorithm.

1. Pick k random centroids.
2. Assign each point to the nearest centroid.
3. Recompute each centroid as the mean of its assigned points.
4. Repeat until assignments stop changing.

Choosing k:

- **Elbow method** plots WCSS (within-cluster sum of squares) vs k. Pick the k where the curve bends – adding more clusters gives diminishing returns.
- **Silhouette score** measures how similar a point is to its own cluster vs others, ranging from -1 to 1 with higher being better.
- Sometimes **domain dictates k**, like three customer tiers.


```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)
print(silhouette_score(X, km.labels_))
```

Pitfalls: K-Means assumes **spherical, equal-sized clusters**; sensitive to scale and outliers; and converges to local minima, so use `n_init=10` and KMeans++ initialisation. For non-spherical clusters, try DBSCAN or Gaussian Mixtures.

Q95. Explain PCA intuitively and the idea of variance retention.

PCA (Principal Component Analysis) compresses many correlated features into a few uncorrelated ones called **principal components**, while keeping as much information (variance) as possible.

Intuition: imagine a 2D scatter plot stretched along a diagonal. PCA rotates the axes so the first new axis (PC1) lies along the direction of **maximum spread**, and PC2 is perpendicular to it. We can then drop PC2 with minimal loss. Each PC has an associated **explained variance ratio**. We typically keep enough components to retain 90–95% of total variance.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
Xs = StandardScaler().fit_transform(X)
pca = PCA(n_components=0.95).fit(Xs)
X_red = pca.transform(Xs)
```

Use cases: speed up training, visualisation with $n=2$ or 3 , reduce multicollinearity, and denoising.

Pitfalls: must **scale features first** since PCA is variance-driven. Components are linear combinations, so **interpretability is lost**. PCA is unsupervised — it doesn't care about the target. For non-linear structure, try t-SNE or UMAP.

Q96. Walk me through accuracy, precision, recall, F1, and ROC-AUC and when each matters.

All come from the **confusion matrix** (TP, FP, FN, TN).

- **Accuracy** = $(TP+TN) / \text{all}$. Misleading on imbalanced data — predicting "not fraud" always can give 99%.

- **Precision** = $TP / (TP + FP)$. "Of those I flagged positive, how many really are?" Use when **false positives** are costly, like flagging a good customer as fraud.
- **Recall (sensitivity)** = $TP / (TP + FN)$. "Of all actual positives, how many did I catch?" Use when **false negatives** are costly – cancer detection, churn, fraud.
- **F1-score** – harmonic mean of precision and recall. Use when classes are imbalanced and you care about both.
- **ROC-AUC** – probability that a random positive ranks higher than a random negative. Threshold-independent and great for comparing models. Less useful when severely imbalanced – prefer **PR-AUC**.

```
from sklearn.metrics import classification_report, roc_auc_score
print(classification_report(y, y_pred))
print(roc_auc_score(y, y_proba))
```

Pitfall: never report only accuracy on imbalanced problems. **Always tie metric to business cost.**

Q97. How do you handle class imbalance? Explain SMOTE, class_weight, and threshold tuning.

Class imbalance, like 1% fraud, makes models biased toward the majority class. Several remedies help.

- **Resampling** – oversample the minority via random oversample or **SMOTE** (Synthetic Minority Oversampling Technique), which interpolates new minority samples between nearest neighbours. Undersampling the majority risks losing information.
- **class_weight='balanced'** – tells the model to penalise mistakes on the minority class more. Available in LogisticRegression, RandomForest, SVM, and others. **No data manipulation needed.**
- **Threshold tuning** – instead of using 0.5 as the probability cutoff, choose a threshold that optimises the business metric, like recall at fixed precision. **Often the simplest, highest-impact fix.**
- **Algorithmic** – `scale_pos_weight` in XGBoost, focal loss in deep nets.
- **Better metrics** – F1, PR-AUC, recall, not accuracy.

```
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE(random_state=42).fit_resample(X_tr, y_tr)
```

Pitfalls: apply SMOTE **only on training fold inside CV** – never on the full dataset, since that causes leakage. Always evaluate on the original imbalanced distribution.

Q98. Why use sklearn Pipeline and ColumnTransformer? Show a small example.

A **Pipeline** chains preprocessing and modelling steps so they run in order. **ColumnTransformer** applies different transforms to different columns. Together they prevent data leakage and make code production-ready.

Benefits: transforms (scalers, encoders, imputers) are fit on the training fold only inside CV, you have a single object to tune via **GridSearchCV**, and it's easy to pickle and deploy.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

num = ['age', 'income']
cat = ['city', 'gender']

num_pipe = Pipeline([('imp', SimpleImputer(strategy='median')),
                     ('sc', StandardScaler())])
cat_pipe = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                     ('oh', OneHotEncoder(handle_unknown='ignore'))])

pre = ColumnTransformer([('n', num_pipe, num), ('c', cat_pipe, cat)])

model = Pipeline([('pre', pre), ('clf', RandomForestClassifier())])
model.fit(X_tr, y_tr)
```

Pitfall: candidates often **fit_transform** on full data before splitting – that **leaks test-set statistics into training**. Always wrap in a **Pipeline**.

Q99. Compare GridSearch, RandomSearch, and Bayesian optimisation for hyperparameter tuning.

All three search the hyperparameter space; they differ in **how they pick candidates**.

- **GridSearchCV** – tries every combination in a defined grid. Exhaustive and reproducible but explodes combinatorially. Good when the grid is small (3–4 hyperparameters with few values).

- **RandomizedSearchCV** – samples random combinations from distributions. Often finds nearly the best result much faster – research shows few hyperparameters matter, and random sampling explores them better. Set `n_iter` to control budget.
- **Bayesian optimisation** (Optuna, Hyperopt, scikit-optimize) – builds a probabilistic model (often a Gaussian Process or TPE) of the score surface and picks the next point intelligently – exploration vs exploitation. Best when each model is expensive to train.

```
from sklearn.model_selection import RandomizedSearchCV
import optuna
```

Practical advice: start with `RandomizedSearch` for quick wins, then refine with Bayesian (Optuna is the industry favourite). Always tune **inside cross-validation**.

Pitfall: tuning on the test set, not setting a random seed, and ignoring early stopping in boosting models.

Q100. What is data leakage? Give common forms and a scenario where train accuracy is 95% but production accuracy is 60%.

Data leakage happens when information unavailable at prediction time sneaks into training, giving inflated metrics that collapse in production.

Common forms:

- **Target leakage** – a feature is a direct consequence of the target – for instance using `loan_status_paid` to predict default.
- **Train-test contamination** – scaling, imputing, or encoding on the entire dataset before splitting.
- **Temporal leakage** – shuffling time-ordered data so future rows influence past predictions.
- **Group leakage** – the same customer or patient appears in both train and test, so the model memorises the entity.
- Lookup tables or aggregations computed using future data.

Scenario fix when train is 95% and prod is 60%:

1. Check if any feature is actually computed *after* the prediction event in real life.
2. Did you fit `StandardScaler`, SMOTE, or target encoder on full data? Move them inside a `Pipeline`.
3. Are duplicate IDs across folds? Use `GroupKFold`.
4. Did you tune on test? Hold a fresh test set.
5. **Distribution shift** between train and prod is also possible – feature drift or label drift.

Pitfall: "too good to be true" results are usually leakage, not genius.

Final Notes for the Candidate

What interviewers actually score you on

1. **Clarity of communication** — can you explain the concept in 60–90 seconds without jargon overload?
2. **Practical judgement** — when *would* you use this? When would you *not*?
3. **Pitfalls awareness** — naming the trap (leakage, scaling order, hidden assumptions) instantly separates good candidates from average ones.
4. **Code reflex** — for NumPy and Pandas, syntax should be muscle memory, not recalled.
5. **Connecting to a project** — anchor every concept to a real dataset you've worked on.

A 14-day prep plan using this document

- **Days 1–2** — NumPy + Pandas (Q1–Q30). Type every code snippet into a notebook.
- **Days 3–4** — Statistics (Q31–Q45). Practice explaining each one out loud as if to a non-technical friend.
- **Days 5–6** — EDA (Q46–Q60). Pick a Kaggle dataset and apply Q47's workflow end-to-end.
- **Days 7–9** — Feature Engineering (Q61–Q75). Build a `Pipeline + ColumnTransformer` from scratch.
- **Days 10–13** — Machine Learning (Q76–Q100). Train each algorithm at least once on a real dataset.
- **Day 14** — Mock interview. Have a friend pick 10 random questions across topics.

Red flags to avoid in interviews

- Saying "*I just use the default*" without knowing what the default is.
- Quoting accuracy on an imbalanced dataset.
- Fitting transformers on the full dataset before splitting.
- Confusing correlation with causation.
- Confusing classification metrics with regression metrics.
- Not knowing what `random_state` is or why it matters.

Closing thought

Interviews at this experience level reward **breadth with depth on the basics**, not exotic algorithms. Master these 100 questions and you will comfortably clear the technical screen at almost every DS, DA, or ML role for 0–3 years experience.

Good luck. Speak with structure, own your pitfalls, and tie everything back to a story.

End of document.